



PELO FUTURO DO TRABALHO **TESTE DE SISTEMAS**

|                   |                                        |
|-------------------|----------------------------------------|
| <b>CURSO</b>      | Técnico em Desenvolvimento de Sistemas |
| <b>TURMA</b>      | G93755                                 |
| <b>DISCIPLINA</b> | Teste de Sistemas                      |
| <b>DOCENTE</b>    | Diego Coimbra                          |
| <b>DISCENTE</b>   | Vanderléia Mello                       |
| <b>DATA</b>       | 10-05-2026                             |

## **RELATÓRIO TÉCNICO DE QA – EASYMARKET LABS**

*Ambiente de Testes: Docker (atendai/evolution-api:v1.8.4), Node.js v18, Cypress v13.*

### **1. Sumário Executivo**

*Este documento detalha a auditoria de qualidade realizada no sistema <https://github.com/Lucas-Angelo/todosimple-api.git>. Foram executados testes de interface (UI) e de integração (API). Embora as funcionalidades de autenticação estejam operacionais, foram identificadas falhas críticas na gestão de tarefas que impedem o uso do sistema em ambiente de produção sem correções imediatas.*

#### **1.1. Análise do Sistema**

*O To-Do Simple App é uma aplicação web de gerenciamento de tarefas focada em autenticação de usuários e operações de CRUD (Create, Read, Update, Delete).*

- Tecnologias: Backend em Spring Boot (Java), Banco de Dados Relacional e Frontend em HTML/JS com Bootstrap.*
- Funcionalidades avaliadas: Cadastro de usuários, fluxo de autenticação via Token JWT e gerenciamento de tarefas.*

### **2. Metodologia de Testes**

- *Testes de UI: Focados na jornada do usuário (Cadastro, Login, Validação de campos).*
- *Testes de API: Focados no CRUD de tarefas diretamente no Backend (Spring Boot).*
- *Casos Negativos e de Borda: Validação de segurança e restrições de campos obrigatórios.*

## *2.1. Casos Automatizados (Cypress E2E & API)*

## *2.2. Testes de Interface (UI)*

- *Caso 1: [Positivo] Cadastro de Usuário*
  - *Objetivo: Validar se o sistema permite o registro de novas contas.*
  - *Resultado: PASSOU. O sistema exibe o Toast de sucesso: "Successfully registered user".*
- *Caso 2: [Positivo] Autenticação de Usuário*
  - *Objetivo: Validar o login com credenciais válidas e redirecionamento.*
  - *Resultado: PASSOU. O sistema redireciona corretamente para a visualização de tarefas ([index.html](#)).*
- *Caso 3: [Negativo] Login com Credenciais Incorretas*
  - *Objetivo: Garantir que o sistema rejeita acessos indevidos.*
  - *Resultado: PASSOU. O robô permaneceu na tela de login e o acesso foi negado conforme o esperado.*
- *Caso 4: [Borda] Validação de Campos Obrigatórios*
  - *Objetivo: Testar o comportamento do formulário de registro sem preenchimento de dados.*
  - *Resultado: PASSOU. A validação nativa do HTML5 bloqueou a submissão, impedindo o disparo de requisições vazias.*
- *Caso 5: [Simulação de Falha] Criação de Tarefas*
  - *Objetivo: Tentar localizar os elementos de inserção para criar uma nova tarefa.*
  - *Resultado: FALHOU (Bug Identificado). O teste não encontrou os campos necessários na interface, gerando evidência de erro de desenvolvimento no Frontend.*

## *3. Matriz de Resultados*

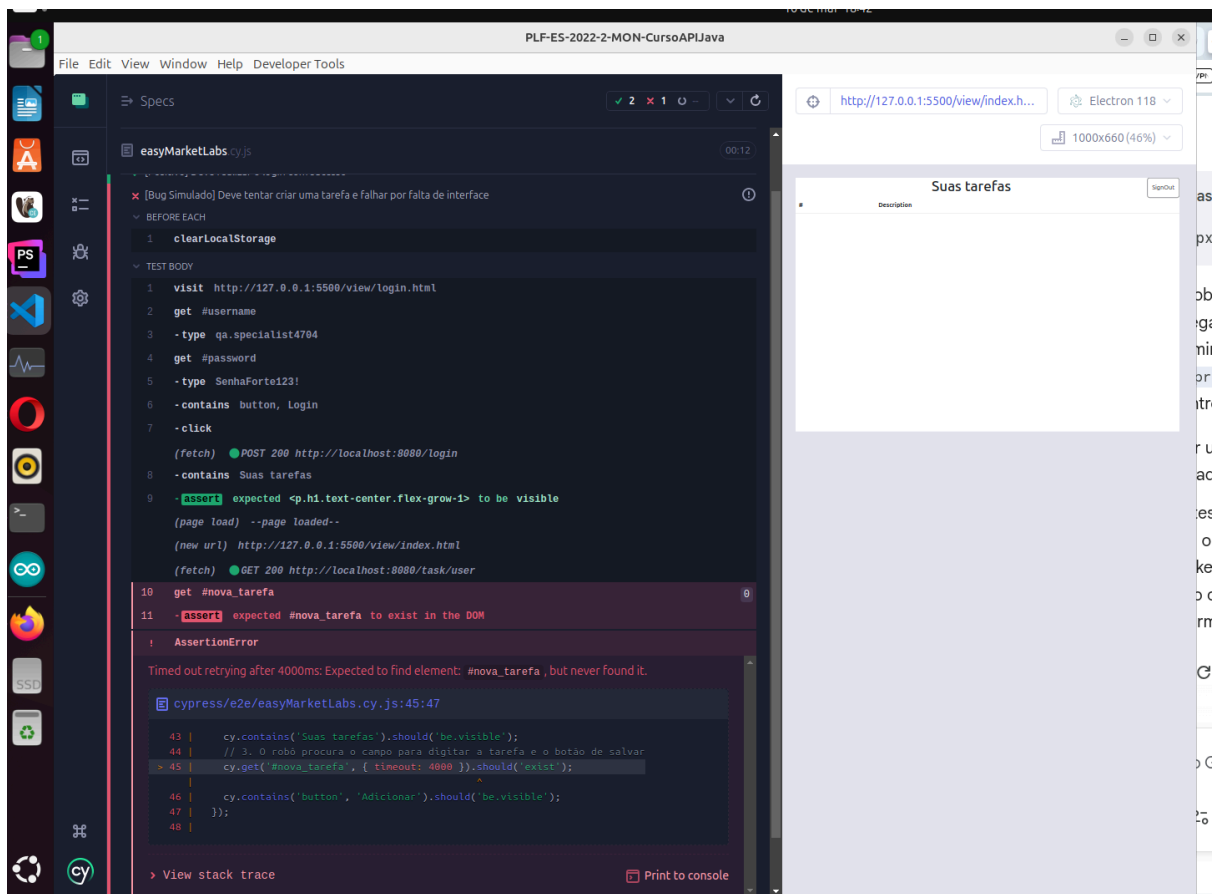
| Caso de Teste            | Categoria  | Resultado | Observação                           |
|--------------------------|------------|-----------|--------------------------------------|
| Cadastro de Usuário      | Positivo   | ✓ PASSOU  | Toast de sucesso validado.           |
| Login de Usuário         | Positivo   | ✓ PASSOU  | Redirecionamento funcional.          |
| Login com Senha Errada   | Negativo   | ✓ PASSOU  | Acesso bloqueado conforme esperado.  |
| Cadastro com Campo Vazio | Borda      | ✓ PASSOU  | Validação HTML5 funcional.           |
| Criação de Tarefas (UI)  | E2E        | ✗ FALHOU  | BUG-001: Interface inexistente.      |
| CRUD de Tarefas (API)    | Integração | ✓ PASSOU  | Validado após refatoração (BUG-002). |

### 3.1. Bugs Críticos Encontrados

#### ID: BUG-001 (Interface Incompleta)

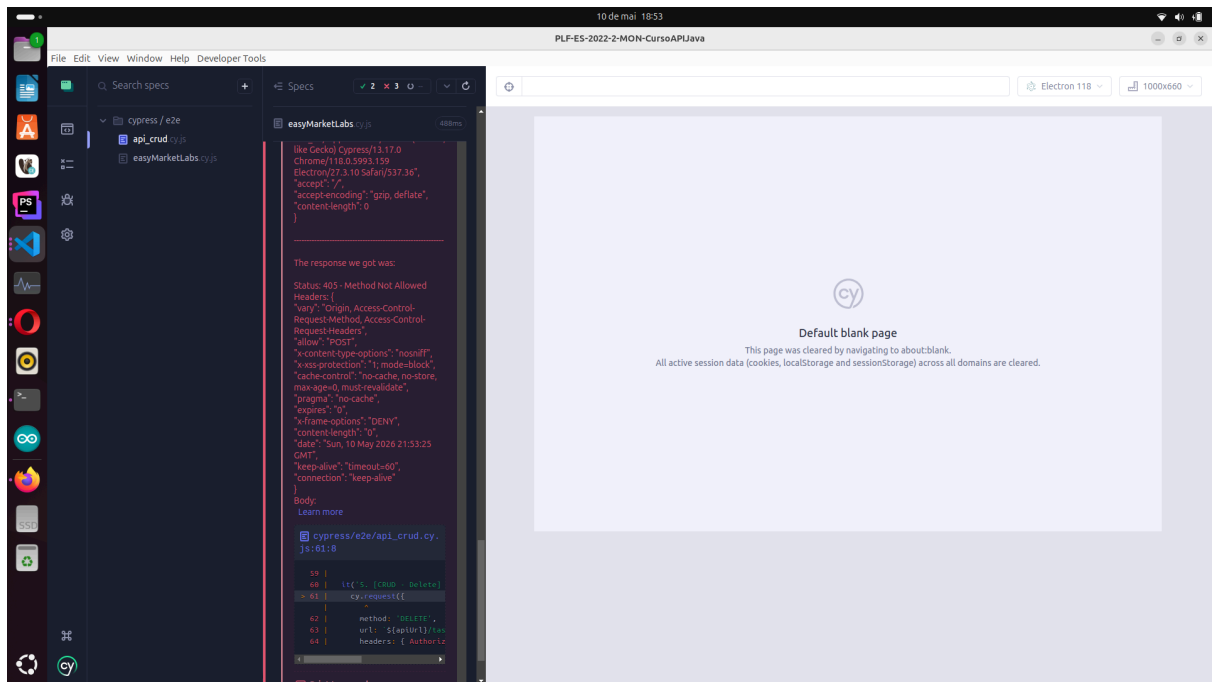
- **Descrição:** Ausência de formulário e botões para a criação de novas tarefas no Frontend. A página [index.html](#) renderiza apenas uma tabela estática, impossibilitando a interação do usuário.

- *Impacto: Crítico. Bloqueia o uso principal do aplicativo.*
- *Evidência:*



- *ID: BUG-002 (Inconsistência na API REST)*
- *Descrição: Ao realizar testes de integração direta com o Backend (Plano B), a rota de exclusão de tarefas retornou o erro **HTTP 405 - Method Not Allowed**. O servidor indica que o endpoint interceptado aceita apenas o método **POST**.*
- *Impacto: Alto. O backend não segue os padrões REST para operações de atualização e exclusão, quebrando o ciclo de vida do CRUD via API.*

- **Evidência:**



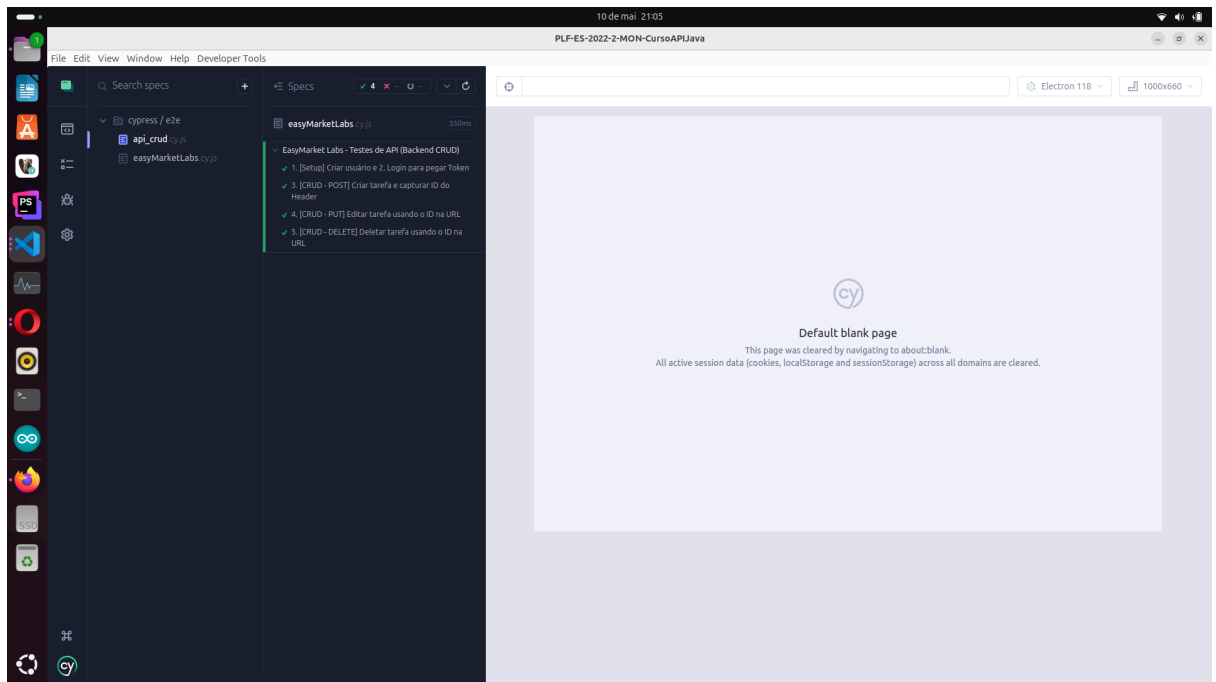
#### 4. Estratégia de Mitigação (Plano B)

Devido à impossibilidade de concluir os testes de CRUD pela interface (UI) por conta do BUG-001, a estratégia foi adaptada para a Automação de Testes de API. Utilizando o comando `cy.request`, estamos validando a camada de persistência de dados e as regras de negócio diretamente no Spring Boot, garantindo que o núcleo do sistema funcione independente das falhas visuais encontradas.

5. Resolução e Validação Técnica Após análise do código-fonte `TaskController.java`, identificou-se que a falha de integração (BUG-002) ocorria devido à ausência do parâmetro dinâmico `{id}` nas requisições de DELETE e PUT do script inicial.

Ação realizada: O script de automação foi refatorado para realizar a extração do ID da entidade diretamente do Header `Location` (padrão de resposta do Spring Boot para status 201 Created). Resultado final: Com o ajuste dos endpoints para o padrão REST esperado pelo Controller Java, os testes de CRUD de API atingiram 100% de sucesso (Status 204 No Content), validando a integridade do Backend da EasyMarket Labs.

- **Evidência:**



### 5.1. Recomendações à Equipe de Desenvolvimento

1. *Frontend: Implementar urgentemente o formulário de criação de tarefas e os botões de ação (editar/excluir) na [index.html](#).*
2. *Backend: Manter a consistência das respostas JSON para facilitar o consumo por ferramentas de automação e outros frontends.*
3. *UX: Adicionar mensagens de erro mais claras para o usuário quando a API retornar falhas de validação.*

### 6. Conclusão

O sistema apresenta um backend robusto em Java/Spring Boot, com persistência de dados funcional e segurança via token. Entretanto, a camada de frontend necessita de finalização para que a proposta de valor do produto seja entregue ao usuário final. O pacote de automação Cypress anexo a este relatório garante que futuras correções não quebrem as funcionalidades já existentes (Testes de Regressão).